

Capability models for Go: An analysis of the most popular dependencies

Group 22

Vilhelm Prytz vprytz@kth.se
Filip Dimitrijevic filipdi@kth.se

May 28, 2026

1 Background

1.1 Capabilities and Capability Models

A *capability* is a permission that grants a program access to a particular system resource, such as the file system, the network, or the ability to execute other processes [1]. A *capability model* describes the set of capabilities that a piece of software can perform, derived from static analysis of its code. Capability models provide a practical way to reason about the *principle of least privilege*: the idea that software should be granted only the minimal set of permissions required to perform its intended function [2].

1.2 Capabilities in Software Dependencies

Modern software relies heavily on third-party dependencies, and a project implicitly inherits the capabilities of every package it imports, often without the developer’s awareness. A *supply-chain attack* is an attack in which an adversary compromises software not by targeting it directly, but by compromising one of the third-party components or dependencies it relies on, so that the malicious code is distributed to every project that includes the affected component [3]. The existence of such attacks against open-source software has shown that dependencies cannot be inherently trusted [2]. Examining the capabilities of a dependency reveals what system resources it can reach, helping to direct security review toward the most privileged code paths and to detect unexpected capability changes that may signal a compromised release [4].

1.3 Capslock

Capslock is a static analysis tool developed by Google that produces capability models for Go packages [2]. It works by following transitive calls from a package’s API down to privileged standard-library operations, and classifies the

results into a set of capability categories. Because the analysis is transitive, a reported capability may originate either in the package’s own code or in one of its dependencies. Capslock is intended to complement vulnerability scanning rather than replace it [2].

The capability categories most relevant to our analysis are summarised in Table 1.

Table 1: Capability categories defined by Capslock, in documentation order [5].

Capability	Description
UNSPECIFIED	Default zero value; cleared to allow deeper analysis
SAFE	Allowlist value that terminates further analysis
FILES	File system access
NETWORK	Network access
RUNTIME	Access to Go runtime state
READ_SYSTEM_STATE	Reading environment or process info
MODIFY_SYSTEM_STATE	Modifying working directory, env vars, or signals
OPERATING_SYSTEM	Catch-all for <code>os</code> package operations
SYSTEM_CALLS	Direct system calls
ARBITRARY_EXECUTION	Type-unsafe or assembler operations
CGO	Executing native code via <code>Cgo</code>
UNANALYZED	Unanalyzable call paths
UNSAFE_POINTER	Use of <code>unsafe.Pointer</code>
REFLECT	Use of reflection
EXEC	Executing other programs

1.4 Go Modules and Semantic Versioning

Go organises code into *modules*, and published releases are identified by version tags following semantic versioning of the form `vMAJOR.MINOR.PATCH` [6]. Major versions signal potentially breaking changes, while minor and patch versions are expected to be backward compatible. This distinction is relevant to our analysis: a capability gained across a major-version rewrite is unsurprising, whereas one gained in a minor or patch release is a more notable change in a package’s privilege footprint.

2 Goal

The goal of this project is to analyze the capability models of the most popular Go libraries and study how their capabilities change over time. We select the top 50 Go libraries by GitHub stars and use Capslock to compute a capability model for every released version of each. By comparing consecutive releases, we aim to answer two questions:

1. How do a package’s capabilities change across its versions?

2. Why do these changes occur?

We restrict our scope to importable libraries and frameworks, since Capslock analyzes Go package call paths and standalone applications expose no meaningful library API to analyze.

This matters for language-based security: capability models show what a dependency can access and flag unexpected changes between releases, helping catch unsafe third-party code. This is a key concern in software supply-chain security.

3 Description of work

We have implemented a complete automated pipeline (fetch data, orchestrate, analyze) as described below.

3.1 Approach

Our work has three stages: selecting the libraries, collecting capability data, and analysing how capabilities change between versions.

We first selected the top Go libraries by GitHub stars using a script that queries the GitHub search API. Since the star ranking mixes libraries with standalone applications and other repositories, we manually filtered the results to keep only importable libraries, giving a final list of 50 (see the curated list in appendix A).

For each library, we cloned the repository and ran Capslock on every release tag, saving the resulting capability model as a JSON file. Runs that failed (for example, older releases that Capslock could not build) were recorded and skipped.

Finally, we loaded all successful results into a single dataset and, for each library, sorted its versions by semantic version and compared the capability set of each release with the previous one. This produced a list of capabilities added and removed between consecutive versions, which forms the basis of our results.

3.2 Code

`fetch_toplist.py` Fetches and filters the GitHub top-500 Go repositories, writing clone URLs to `toplist_repositories.txt`.

`curated_toplist_repositories.txt` The manually curated final list of 50 library repositories used as input to the pipeline.

`Dockerfile` Alpine image with Go and Capslock pre-installed, used as the build environment for each analysis run.

`orchestrator.py` Selects release tags and runs the Docker container for each, writing Capslock JSON output to `results/`.

`analysis/notebook.ipynb` Loads the collected JSON files and produces the capability-diff tables and plots.

3.3 Running the analysis

Before you start, ensure you have Docker running. To run the analysis, start by cloning the GitHub project.¹

Then, to get a list of top GitHub repositories, we recommend to create a personal access token.² This is not a requirement, but you may run into rate limits when fetching data from the GitHub API. Using an authenticated token, this limit is significantly higher. Export your token as an environment variable called `GITHUB_TOKEN`.

```
export GITHUB_TOKEN="your token here"
```

If you have `direnv`³, a Python Virtual Environment will be created automatically. If you do not, it is recommended to create one. Install the requirements for running the scripts and the Jupyter Notebook.

```
pip install -r requirements.txt
pip install -r analysis/requirements.txt
```

Run the `fetch_toplist.py` to fetch a list of GitHub repositories.

```
python fetch_toplist.py
```

The list is saved in `toplist_repositories.txt`. The analysis is performed on repositories listed in `curated_toplist_repositories.txt` which is committed in the repository.

Run the `orchestrator.py` script to perform the analysis inside Docker. This can take some time.

```
python orchestrator.py
```

Once the script has finished, start Jupyter Lab in the analysis folder.

```
cd analysis
jupyter lab .
```

Run the notebook `notebook.ipynb` to perform the analysis and get the results.

¹<https://github.com/vilhelmprytz/dd2525-project>

²<https://github.com/settings/tokens>

³<https://direnv.net/>

4 Results

4.1 Heatmap Overview

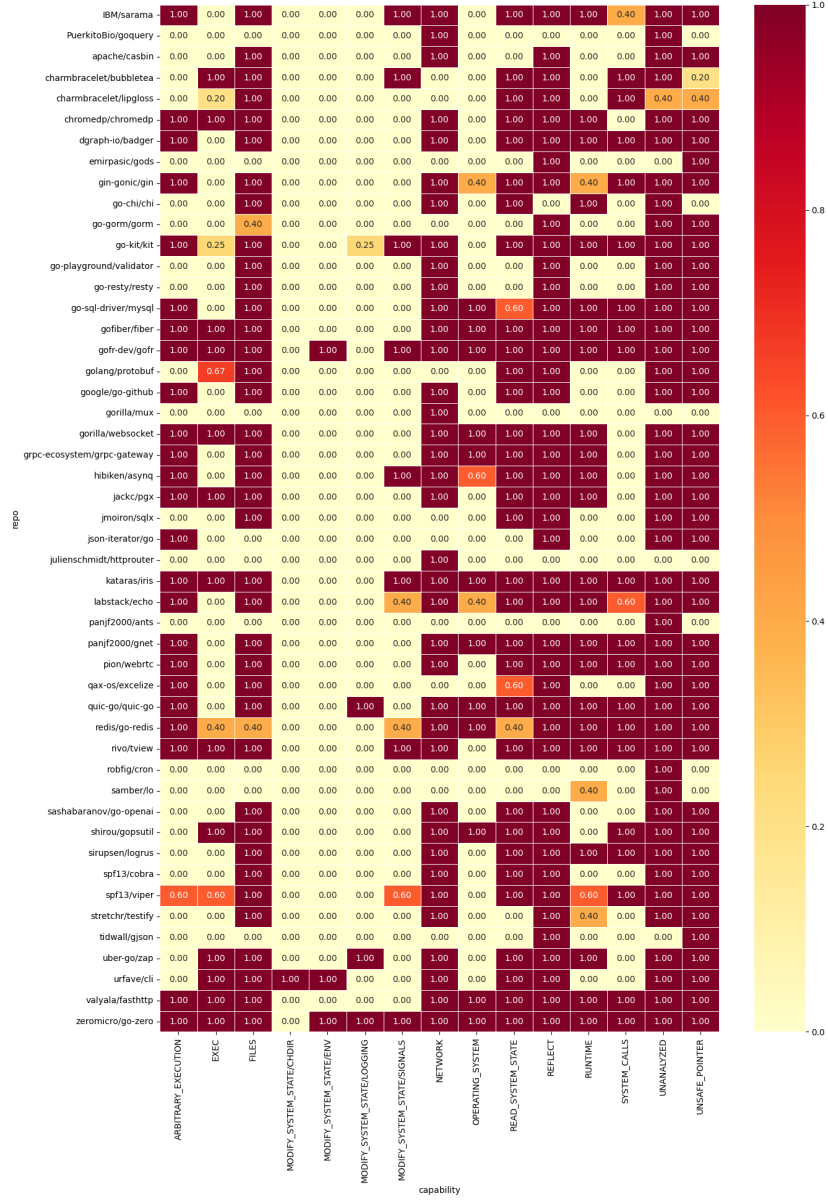


Figure 1: Heatmap showing capability fraction.

In Figure 1 (source⁴), we can see an heatmap of the analyzed repositories and fractions of how much of the analyzed releases contain the specific capability. This means that cells with the value of 0.00 means that none of the analyzed releases contain any code that uses this specific capability. A value of 1.00 means that all analyzed releases uses this capability. Hence, the interesting cells are the ones fulfilling $0.00 < x < 1.00$, since this indicates a change in capabilities across releases.

4.2 Capability Changes

From each change in the result, we looked up the commit history for each release diff. See Appendix B for full result list.

The following releases were skipped due to build failures. The analysis includes 49 repositories with a total of 213 releases.

```

Skipping failed ../results/beego_beego at tag v2.3.10
Skipping failed ../results/beego_beego at tag v2.0.7
Skipping failed ../results/beego_beego at tag v1.12.14
Skipping failed ../results/beego_beego at tag v2.2.2
Skipping failed ../results/beego_beego at tag v2.1.6
Skipping failed ../results/go-chi_chi at tag v4.1.3
Skipping failed ../results/go-kit_kit at tag v0.9.0
Skipping failed ../results/golang_protobuf at tag v1.1.0
Skipping failed ../results/golang_protobuf at tag v1.2.0
Skipping failed ../results/gorilla_mux at tag v1.6.2
Skipping failed ../results/gorilla_mux at tag v1.5.0
Skipping failed ../results/gorilla_mux at tag v1.4.0
Skipping failed ../results/gorilla_websocket at tag v1.1.0
Skipping failed ../results/gorilla_websocket at tag v1.3.0
Skipping failed ../results/gorilla_websocket at tag v1.2.0
Skipping failed ../results/json-iterator_go at tag 1.0.6
Skipping failed ../results/julienschmidt_httprouter at tag v1.1.0
Skipping failed ../results/julienschmidt_httprouter at tag v1.0.0
Skipping failed ../results/julienschmidt_httprouter at tag v1.2.0
Skipping failed ../results/kataras_iris at tag v12.1.8
Skipping failed ../results/kataras_iris at tag v12.0.1
Skipping failed ../results/kataras_iris at tag v0.0.2
Skipping failed ../results/robfig_cron at tag v1.1.0
Skipping failed ../results/robfig_cron at tag v1.0.0
Skipping failed ../results/robfig_cron at tag v1.2.0
Loaded 49 repos with total of 213 releases

```

All changes are summarized in Table 2, which lists each library, its version transition, whether it was a major or minor bump, and the specific capabilities added or removed (defined in Table 1).

⁴<https://github.com/vilhelmprytz/dd2525-project/blob/main/analysis/notebook.ipynb>

Table 2: Summary of capability changes between analysed releases. “+” marks an added capability, “-” a removed one.

Library	Transition	Bump	Change
IBM/sarama	v1.47.0 → v1.48.2	minor	+SYSTEM_CALLS
charmbracelet/bubbletea	v1.3.10 → v2.0.6	major	+UNSAFE_POINTER
charmbracelet/lipgloss	v1.0.0 → v1.1.0	minor	+UNSAFE_POINTER, +UNANALYZED
charmbracelet/lipgloss	v1.1.0 → v2.0.3	major	+EXEC
gin-gonic/gin	v1.10.1 → v1.11.0	minor	+RUNTIME, +OPERATING_SYSTEM
go-gorm/gorm	v1.26.1 → v1.30.5	minor	+FILES
go-kit/kit	v0.10.0 → v0.11.0	minor	-EXEC, -MODIFY_SYSTEM_STATE/LOGGING
go-sql-driver/mysql	v1.7.1 → v1.8.1	minor	+READ_SYSTEM_STATE
golang/protobuf	v1.3.5 → v1.4.3	minor	+EXEC
hibiken/async	v0.23.0 → v0.24.1	minor	+OPERATING_SYSTEM
labstack/echo	v4.15.2 → v5.0.4	major	+MODIFY_SYSTEM_STATE/SIGNALS, +OPERATING_SYSTEM, -SYSTEM_CALLS
qax-os/excelize	v2.8.1 → v2.9.1	minor	-READ_SYSTEM_STATE
redis/go-redis	v9.17.3 → v9.18.0	minor	+EXEC, +FILES, +READ_SYSTEM_STATE, +MOD- IFY_SYSTEM_STATE/SIGNALS
samber/lo	v1.51.0 → v1.52.0	minor	+RUNTIME
spf13/viper	v1.19.0 → v1.20.1	minor	-ARBITRARY_EXECUTION, -EXEC, -MODIFY_SYSTEM_STATE/SIGNALS, -RUNTIME
stretchr/testify	v1.9.0 → v1.10.0	minor	+RUNTIME

4.3 Discussion

Returning to our two research questions: across the 49 libraries and 213 releases, most libraries showed no change in their capabilities, and only 16 changed at all. Every change we found had a clear cause, such as a new feature, a major version rewrite, a dependency update, or test and build code in the repository. We found no suspicious or unexplained changes in capabilities.

The most interesting finding is that the most severe capabilities almost never came from code that a user of the library actually imports. `redis/go-redis` adds four capabilities in one minor release, but the call paths show that `EXEC` and `READ_SYSTEM_STATE` come from a test package (`maintnotifications/e2e`), not from the client itself. `golang/protobuf` gained `EXEC` only from an internal code-generation tool, and `hibiken/async` gained `OPERATING_SYSTEM` from a test helper. These capabilities still show up even though a user of the library never runs the packages they come from.

An interesting capability to look at is the `NETWORK` capability. We can see from our results that this capability was not modified in any of the releases, which is a good sign. A sudden introduction of the `NETWORK` capability in a non-networking library could be a red flag that something suspicious is happening. This was not the case in any of the analyzed repositories.

5 Contributions

We have done all the work together, either in person or over Zoom/Discord. We have pair-programmed and taken turns in being driver and navigator/observer. Some focus areas:

Vilhelm: Focused on the scripts and the use of Capslock together with the GitHub token. Also analysed and structured the results, in particular the heatmap, and planned the structure and content of the report.

Filip: Manually selected and verified the 50 repositories as suitable for analysis, studied the Capslock documentation to identify and understand the capability categories, analysed the results, and planned the structure and content of the report.

6 AI-powered tools

AI-powered tools were used as a search engine, and for formatting in L^AT_EX. It was also used to fix some spelling mistakes and grammar in the report. All code was written by us; no code or any parts of any solutions were generated using AI, they were written by us, the authors.

References

- [1] J. B. Dennis and E. C. Van Horn, “Programming semantics for multi-programmed computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966. DOI: 10.1145/365230.365252.
- [2] Google, *Capslock: A capability analysis cli for go packages*, <https://github.com/google/capslock>, Accessed: 2026-05-21, 2026.
- [3] M. Ohm, H. Plate, A. Sykosch, and M. Meier, “Backstabber’s knife collection: A review of open source software supply chain attacks,” in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. Lecture Notes in Computer Science, vol. 12223, Springer, 2020, pp. 23–43. DOI: 10.1007/978-3-030-52683-2_2.
- [4] J. Murphy and J. Kern, *Capslock: What is your code really capable of?* <https://security.googleblog.com/2023/09/capslock-what-is-your-code-really.html>, Google Security Blog. Accessed: 2026-05-21, 2023.
- [5] Google, *Capslock capabilities documentation*, <https://github.com/google/capslock/blob/main/docs/capabilities.md>, Accessed: 2026-05-21, 2026.
- [6] The Go Authors, *Go modules reference*, <https://go.dev/ref/mod>, Accessed: 2026-05-21, 2026.

A Curated top list

Source on GitHub. ⁵

<https://github.com/gin-gonic/gin.git>
<https://github.com/spf13/cobra.git>
<https://github.com/charmbracelet/bubbletea.git>
<https://github.com/go-gorm/gorm.git>
<https://github.com/gofiber/fiber.git>
<https://github.com/zeromicro/go-zero.git>
<https://github.com/beego/beego.git>
<https://github.com/labstack/echo.git>
<https://github.com/spf13/viper.git>
<https://github.com/go-kit/kit.git>
<https://github.com/stretchr/testify.git>
<https://github.com/sirupsen/logrus.git>
<https://github.com/kataras/iris.git>
<https://github.com/gorilla/websocket.git>
<https://github.com/uber-go/zap.git>
<https://github.com/urfave/cli.git>
<https://github.com/valyala/fasthttp.git>
<https://github.com/go-chi/chi.git>
<https://github.com/redis/go-redis.git>
<https://github.com/gorilla/mux.git>
<https://github.com/gofr-dev/gofr.git>
<https://github.com/samber/lo.git>
<https://github.com/qax-os/excelize.git>
<https://github.com/apache/casbin.git>
<https://github.com/go-playground/validator.git>
<https://github.com/grpc-ecosystem/grpc-gateway.git>
<https://github.com/jmoiron/sqlx.git>
<https://github.com/emirpasic/gods.git>
<https://github.com/julienschmidt/httprouter.git>
<https://github.com/pion/webrtc.git>
<https://github.com/dgraph-io/badger.git>
<https://github.com/tidwall/gjson.git>
<https://github.com/go-sql-driver/mysql.git>
<https://github.com/PuerkitoBio/goquery.git>
<https://github.com/panjf2000/ants.git>
<https://github.com/robfig/cron.git>
<https://github.com/json-iterator/go.git>
<https://github.com/rivo/tview.git>
<https://github.com/jackc/pgx.git>
<https://github.com/hibiken/asynq.git>
<https://github.com/chromedp/chromedp.git>

⁵<https://github.com/vilhelmprytz/dd2525-project>

```

https://github.com/IBM/sarama.git
https://github.com/shirou/gopsutil.git
https://github.com/go-resty/resty.git
https://github.com/quic-go/quic-go.git
https://github.com/charmbracelet/lipgloss.git
https://github.com/google/go-github.git
https://github.com/panjf2000/gnet.git
https://github.com/sashabaranov/go-openai.git
https://github.com/golang/protobuf.git

```

B Full result

The log below overflows somewhat in L^AT_EX, see the GitHub page for more details if needed. ⁶

```

IBM/sarama: v1.47.0 → v1.48.2
  ADDED:  {'SYSTEM_CALLS'}
    [SYSTEM_CALLS] github.com/IBM/sarama
      via: github.com/IBM/sarama.getTCPConnSockError$1 golang.org/x/sys/unix.GetsockoptInt
charmbracelet/bubbletea: v1.3.10 → v2.0.6
  ADDED:  {'UNSAFE_POINTER'}
    [UNSAFE_POINTER] charm.land/bubbletea/v2
      via: (*charm.land/bubbletea/v2.cursedRenderer).flush (*github.com/charmbracelet/ultra
charmbracelet/lipgloss: v1.0.0 → v1.1.0
  ADDED:  {'UNSAFE_POINTER', 'UNANALYZED'}
    [UNANALYZED] github.com/charmbracelet/lipgloss
      via: (github.com/charmbracelet/lipgloss.Style).Render github.com/charmbracelet/x/celll
    [UNSAFE_POINTER] github.com/charmbracelet/lipgloss
      via: (github.com/charmbracelet/lipgloss.Style).Render github.com/charmbracelet/x/celll
charmbracelet/lipgloss: v1.1.0 → v2.0.3
  ADDED:  {'EXEC'}
    [EXEC] charm.land/lipgloss/v2/compat
      via: charm.land/lipgloss/v2/compat.init github.com/charmbracelet/colorprofile.Detect g
gin-gonic/gin: v1.10.1 → v1.11.0
  ADDED:  {'OPERATING_SYSTEM', 'RUNTIME'}
    [OPERATING_SYSTEM] github.com/gin-gonic/gin
      via: (*github.com/gin-gonic/gin.Engine).LoadHTMLFS (*html/template.Template).ParseFS h
    [RUNTIME] github.com/gin-gonic/gin
      via: (*github.com/gin-gonic/gin.Engine).RunQUIC github.com/quic-go/quic-go/http3.Liste
go-gorm/gorm: v1.26.1 → v1.30.5
  ADDED:  {'FILES'}
    [FILES] gorm.io/gorm/logger
      via: (*gorm.io/gorm/logger.slogLogger).Info (*log/slog.Logger).InfoContext (*log/slog
go-kit/kit: v0.10.0 → v0.11.0

```

⁶<https://github.com/vilhelmprytz/dd2525-project/blob/main/analysis/notebook.ipynb>

```

    REMOVED: {'EXEC', 'MODIFY_SYSTEM_STATE/LOGGING'}
go-sql-driver/mysql: v1.7.1 → v1.8.1
    ADDED: {'READ_SYSTEM_STATE'}
        [READ_SYSTEM_STATE] github.com/go-sql-driver/mysql
            via: github.com/go-sql-driver/mysql.encodeConnectionAttributes os.Getpid
golang/protobuf: v1.3.5 → v1.4.3
    ADDED: {'EXEC'}
        [EXEC] github.com/golang/protobuf/internal/cmd/generate-alias
            via: github.com/golang/protobuf/internal/cmd/generate-alias.main os/exec.Command
hibiken/async: v0.23.0 → v0.24.1
    ADDED: {'OPERATING_SYSTEM'}
        [OPERATING_SYSTEM] github.com/hibiken/async/internal/testutil
            via: github.com/hibiken/async/internal/testutil.FlushDB (github.com/redis/go-redis/v9)
labstack/echo: v4.15.2 → v5.0.4
    ADDED: {'OPERATING_SYSTEM', 'MODIFY_SYSTEM_STATE/SIGNALS'}
        [MODIFY_SYSTEM_STATE/SIGNALS] github.com/labstack/echo/v5
            via: (*github.com/labstack/echo/v5.Echo).Start os/signal.NotifyContext
        [OPERATING_SYSTEM] github.com/labstack/echo/v5/middleware
            via: github.com/labstack/echo/v5/middleware.listdir io/fs.ReadDir slices.SortFunc[[]io]
    REMOVED: {'SYSTEM_CALLS'}
gax-os/excelize: v2.8.1 → v2.9.1
    REMOVED: {'READ_SYSTEM_STATE'}
redis/go-redis: v9.17.3 → v9.18.0
    ADDED: {'READ_SYSTEM_STATE', 'EXEC', 'MODIFY_SYSTEM_STATE/SIGNALS', 'FILES'}
        [EXEC] github.com/redis/go-redis/v9/maintnotifications/e2e
            via: (*github.com/redis/go-redis/v9/maintnotifications/e2e.ProxyFaultInjectorServer).S
        [FILES] github.com/redis/go-redis/v9
            via: (*github.com/redis/go-redis/v9.InfoCmd).readReply (*bufio.Scanner).Scan (*os.File)
        [MODIFY_SYSTEM_STATE/SIGNALS] github.com/redis/go-redis/v9
            via: github.com/redis/go-redis/v9.cmdString (os/signal.signalError).Error
        [READ_SYSTEM_STATE] github.com/redis/go-redis/v9/maintnotifications/e2e
            via: github.com/redis/go-redis/v9/maintnotifications/e2e.DebugE2E os.Getenv
samber/lo: v1.51.0 → v1.52.0
    ADDED: {'RUNTIME'}
        [RUNTIME] github.com/samber/lo/it
            via: github.com/samber/lo/it.Zip2$1 iter.Pull$3 runtime.Goexit
spf13/viper: v1.19.0 → v1.20.1
    REMOVED: {'EXEC', 'MODIFY_SYSTEM_STATE/SIGNALS', 'ARBITRARY_EXECUTION', 'RUNTIME'}
stretchr/testify: v1.9.0 → v1.10.0
    ADDED: {'RUNTIME'}
        [RUNTIME] github.com/stretchr/testify/assert
            via: (*github.com/stretchr/testify/assert.CollectT).FailNow runtime.Goexit

```